

This technical report provides the detailed pseudocode of the proposed greedy algorithms and the proofs of the theoretical results. Algorithm 1 (Followers) describes the procedure for computing the follower set induced by an anchored edge. Algorithm 2 (GIA) presents the greedy incremental anchoring strategy, which iteratively selects the edge-increment action with the largest follower gain per unit budget. Algorithm 3 (GPA) accelerates GIA by using candidate reduction, follower-ratio upper bounds, and intermediate-result reuse, while preserving the same greedy selection criterion.

A Pseudo code of Followers

Algorithm 1: Followers(G, e, δ_e, s, C_s)

```

1 PQ ← queue(); // sort by coreness-layer ascending order
2  $\mathcal{F} \leftarrow \emptyset$ ;
3  $w(e) \leftarrow w(e) + \delta_e$ ;
4 for each  $x \in \{u, v\}$  if  $x \notin C_s$  do
5   PQ.add( $x$ );
6 while PQ  $\neq \emptyset$  do
7    $x \leftarrow \text{PQ.pop}()$ ;
8    $w^+(x) \leftarrow \text{SupportWeightedDegree}(G, x)$ ;
9   if  $w^+(x) \geq s$  then
10     $\mathcal{F} \leftarrow \mathcal{F} \cup \{x\}$ ;
11    for each  $y \in N(x, G)$  and  $x \prec y$  do
12      if  $y \notin C_s$  and  $y \notin \mathcal{F}$  then
13        PQ.add( $y$ );
14   else
15     Q ← queue();
16     Q.push( $x$ );
17     while Q  $\neq \emptyset$  do
18        $y \leftarrow \text{Q.pop}()$ ;
19       if  $y \in \mathcal{F}$  then
20          $\mathcal{F} \leftarrow \mathcal{F} \setminus \{y\}$ ;
21       for each  $z \in N(y, G) \cap \mathcal{F}$  do
22          $w^+(z) \leftarrow w^+(z) - w(y, z)$ ;
23         if  $w^+(z) < s$  then
24           Q.push( $z$ );
25  $w(e) \leftarrow w(e) - \delta_e$ ;
26 return  $\mathcal{F}$ 

```

Algorithm description. Algorithm 1 computes the follower set induced by an anchored edge via a localised simulation of the s -core peeling process. Starting from the anchored endpoints, the algorithm explores only nodes that are reachable through neighbours with higher coreness-layer pairs, following the upstairs order established by the onion-layer structure. A node is retained as a follower if its support weighted degree meets the s -core threshold; otherwise, it is discarded. Discarding a node may reduce the support of neighbouring retained followers, potentially triggering further removals. This cascading update exactly mirrors the behaviour of the s -core decomposition, but is restricted to the structurally affected region. The process terminates when no further admissible nodes remain, and the surviving candidates constitute the final follower set.

B Pseudo code of GIA

Algorithm description. Algorithm 2 presents the baseline greedy incremental anchoring procedure of GIA. Starting from the initial s -core, the algorithm iteratively evaluates candidate edges whose endpoints are not both contained in the current s -core. For each candidate edge, GIA determines the required weight increment using the

Algorithm 2: GIA(G, b, s)

```

1  $G' \leftarrow G, A \leftarrow \emptyset, \text{sum} \leftarrow 0$ ;
2  $C_s \leftarrow s\text{-core}(G')$ ;
3 while  $\text{sum} < b$  do
4    $E^* \leftarrow \binom{V}{2} \setminus \binom{C_s}{2}$ ;
5    $e^* \leftarrow \text{none}, \delta^* \leftarrow 0, \text{max\_FR} \leftarrow 0$ ;
6   for each  $e \in E^*$  do
7      $\delta_e \leftarrow \text{computeDelta}(G', s, e)$ ;
8     if  $\text{sum} + \delta_e \leq b$  then
9        $\mathcal{F}(e, \delta_e) \leftarrow \text{Followers}(G', e, \delta_e, s, C_s)$ ;
10       $\text{FR}(e) \leftarrow |\mathcal{F}(e, \delta_e)| / \delta_e$ ;
11      if  $\text{max\_FR} < \text{FR}(e)$  then
12         $e^* \leftarrow e, \delta^* \leftarrow \delta_e$ ;
13         $\text{max\_FR} \leftarrow \text{FR}(e)$ ;
14    $w_{G'}(e^*) \leftarrow w_{G'}(e^*) + \delta^*$ ;
15    $\text{sum} \leftarrow \text{sum} + \delta^*$ ;
16    $A \leftarrow A \cup (e^*, \delta^*)$ ;
17    $C_s \leftarrow s\text{-core}(G')$ ;
18 return  $A$ 

```

CGI rule, computes the induced follower set, and evaluates the corresponding follower ratio. At each iteration, the edge achieving the largest follower ratio is selected, its weight is increased accordingly, and the s -core is updated. This process repeats until the available budget is exhausted, yielding a greedy baseline for comparison.

C Pseudo code of GPA

Algorithm description. Algorithm 3 presents the pseudocode of GPA, an accelerated variant of GIA that reduces the candidate search space while preserving the same greedy selection criterion. GPA exploits the locality of follower propagation by organising nodes outside the s -core into connected components and maintaining, for each component, the best edge-weight increment candidate pair identified so far. At each iteration, GPA selects the current best candidate across all components. It then explores candidate edges whose endpoints belong to different components, guided by node-level and edge-level upper bounds that allow early termination and safe pruning. Only candidate pairs that cannot be ruled out by these bounds are evaluated using the same follower computation as in GIA. After applying the selected edge reinforcement, GPA updates the component structure. If the reinforced edge connects two distinct components, they are merged and their best candidate is recomputed; otherwise, only the affected component is updated. In both cases, the s -core is recomputed locally within the affected components. This process repeats until the budget is exhausted, yielding the same greedy outcome as GIA with fewer candidate evaluations.

D Proof of Theorem 1

PROOF. We reduce the Maximum Coverage (MC) problem, which is known to be NP-hard unless $P = NP$ [2], to the WASCO problem. The MC problem is defined as follows: given a collection of sets $\{T_1, T_2, \dots, T_t\}$ containing elements from a universe $\{e_1, e_2, \dots, e_d\}$

Algorithm 3: GPA(G, b, s)

```

1  $G' \leftarrow G, A \leftarrow \emptyset, \text{sum} \leftarrow 0;$ 
2  $L \leftarrow \text{list}(), M \leftarrow \text{map}();$  // maps each component to its
   best  $(e, \delta_e, FR)$ 
3  $C_s \leftarrow s\text{-core}(G');$ 
4  $M \leftarrow \text{buildComponents}(G', b, s);$ 
5 while  $\text{sum} < b$  do
6    $e^*, \delta^*, \text{max\_FR} \leftarrow \arg \max_{(e, \delta_e, FR) \in M} FR;$ 
7   for  $u \in V \setminus C_s$  and  $\text{sum} + (s - c(u)) \leq b$  do
8      $U(u) \leftarrow \text{UpperBound}(G', u, s);$ 
9      $L.append(u);$ 
10   $\text{sort}(L);$  // by upper bound decreasing order
11  for  $u \in L$  do
12    if  $\text{max\_FR} > 2 \cdot U(u)$  then
13      break;
14    for  $v \in L$  s.t.  $v$  appears after  $u$  do
15      if  $u.\text{component} = v.\text{component}$  then
16        continue;
17      if  $\text{max\_FR} > U(u) + U(v)$  then
18        break;
19      if  $\text{max\_FR} > U(u, v)$  then
20        continue;
21      else
22         $e \leftarrow (u, v);$ 
23         $\delta_e \leftarrow \text{computeDelta}(G', s, e);$ 
24        if  $\text{sum} + \delta_e \leq b$  then
25           $\mathcal{F}(e, \delta_e) \leftarrow \text{Followers}(G', e, \delta_e, s, C_s);$ 
26           $FR(e) \leftarrow |\mathcal{F}(e, \delta_e)| / \delta_e;$ 
27          if  $\text{max\_FR} < FR(e)$  then
28             $e^* \leftarrow e, \delta^* \leftarrow \delta_e;$ 
29             $\text{max\_FR} \leftarrow FR(e);$ 
30   $w_{G'}(e^*) \leftarrow w_{G'}(e^*) + \delta^*;$ 
31   $\text{sum} \leftarrow \text{sum} + \delta^*;$ 
32   $A \leftarrow A \cup (e^*, \delta^*);$ 
33   $\text{updateComponents}(G', b, s, \text{sum}, M);$ 
34 return  $A$ 

```

and a budget b' , the objective is to select b' sets such that the number of unique elements covered is maximised.

To construct an instance of the WASCO problem, consider an arbitrary instance I_{MC} of the MC problem with sets $\{T_1, \dots, T_t\}$ and elements $\{e_1, \dots, e_d\} = \bigcup_{i=1}^t T_i$, as illustrated in the top of Figure 1. We define an instance $I_{WASCO} = (G, s, b)$ as follows:

- The graph G consists of three components: a part M representing the sets T_i , a part N representing the elements e_j , and a set of $(d+2)$ -cliques.
- For each set T_i , create a corresponding set of nodes M_i in M , where $M_i = \{u_{i,1}, u_{i,2}, \dots, u_{i,d+2}\}$. The nodes $\{u_{i,1}, \dots, u_{i,d+1}\}$ form a $(d+1)$ -clique, and $u_{i,d+2}$ is connected to $\{u_{i,1}, \dots, u_{i,d}\}$.
- For each element e_j , create a corresponding node v_j in N . Add an edge between v_j and $u_{i,j}$ for every set T_i such that $e_j \in T_i$.

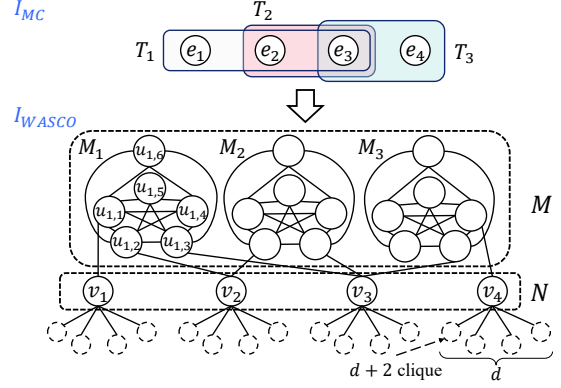


Figure 1: Proof of NP-hardness

- For each v_j , add d disjoint $(d+2)$ -cliques, and connect v_j to one node in each clique.
- Set $s = d + 1$ and $b = b'$.
- Every edge in G has a weight of 1.

This construction can clearly be completed in polynomial time. Initially, during the s -core decomposition, all nodes in M and N are removed, leaving only the $(d+2)$ -cliques. This happens because:

- Nodes $u_{i,d+1}$ and $u_{i,d+2}$ in M_i have degree d , which is less than $s = d + 1$, and are therefore removed.
- Removing $u_{i,d+1}$ and $u_{i,d+2}$ causes the remaining nodes $u_{i,j}$ in M_i to fail the s -core condition, as their degree drops below s .
- All v_j nodes in N are similarly removed because their connections to M_i are lost.

As a result, only the $(d+2)$ -cliques remain in the s -core after this initial decomposition. To maximise the s -core size, an optimal solution is to add an edge between $u_{i,d+1}$ and $u_{i,d+2}$ in M_i . Adding this edge corresponds to selecting the set T_i in the MC problem. Note that another optimal solution adds edges between distinct sets in M (e.g., $(u_{i,d+2}, u_{j,d+1})$ and $(u_{j,d+2}, u_{i,d+1})$), which can always be replaced by the intra-set connection since both strategies yield the exact same number of followers. When the edge is added:

- All nodes in M_i are included in the s -core because the degree of $u_{i,d+1}$ and $u_{i,d+2}$ becomes $d + 1$.
- All nodes v_j connected to M_i are also included in the s -core, as their degree conditions are satisfied.

Since the optimal way to maximise the s -core is to add edges corresponding to specific T_i sets so as to include as many e_i elements as possible in the s -core, this process directly mirrors the objective of the MC problem to select sets that maximise the coverage of elements. Thus, selecting M_i that are more densely connected to uncovered v_j aligns with the optimisation goals of the MC problem.

Therefore, the MC problem is reducible to the WASCO problem, implying that $I_{MC} \leq_P I_{WASCO}$. As the MC problem is NP-hard, this proves the NP-hardness of the WASCO problem. $\square$

E Proof of Theorem 2

PROOF. We give a gap-preserving reduction from the Maximum Coverage (MC) problem. It is known that, for any $\varepsilon > 0$, MC cannot

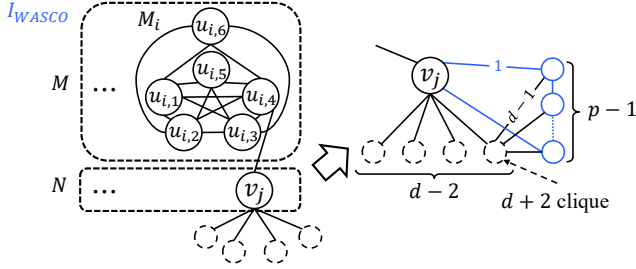


Figure 2: Gadget for the inapproximability reduction

be approximated within a factor of $(1 - 1/e + \varepsilon)$ in polynomial time, unless $P = NP$ [1].

We start from the construction in the proof of Theorem 1 and apply the same two modifications to each node in part N (Figure 2). For each element node v_j , we (i) reduce the number of disjoint $(d+2)$ -cliques attached to v_j from d to $d-2$, and (ii) replace v_j by a cycle on p nodes by adding $p-1$ new nodes connected to v_j with edges of weight 1. Moreover, each of the $p-1$ new cycle nodes is connected to one node of a (randomly chosen) attached $(d+2)$ -clique by an edge of weight $d-1$. We set $s = d+1$ as in Theorem 1. Let p be an integer chosen as a polynomially bounded function of the MC instance size; its value will be specified at the end of the proof.

Let G be the resulting WASCO instance, and let $f(A) = |C_s(G_A)|$ denote the size of the s -core after applying an anchoring solution A with total budget b . As in Theorem 1, without any anchoring, all nodes in M and N are removed by the s -core decomposition, and only the union of the $(d+2)$ -cliques remains in the s -core. Denote by B the number of nodes in this always-retained clique part; B is fully determined by the MC instance and is independent of A .

Now consider any feasible anchoring solution A that uses exactly b budget units (if it uses fewer, we can add arbitrary weight increments on edges between already-retained clique nodes to exhaust the budget without changing the s -core size). By the same argument as in Theorem 1, anchoring an edge within a set gadget M_i corresponds to selecting the set T_i , which makes all $d+2$ nodes of M_i enter the s -core; since exactly b sets are selected, this contributes an additive term $(d+2)b$ to $f(A)$.

Let $\text{cov}(A)$ be the number of elements covered by the corresponding b selected sets in the MC instance. By construction, $\text{cov}(A)$ is exactly the number of element nodes v_j that enter the s -core under A . We next show that if a node v_j enters the s -core, then all p nodes on the cycle containing v_j also enter the s -core. Indeed, once v_j is retained, every other cycle node has weighted degree $2 + (d-1) = d+1 = s$, coming from its two cycle neighbours (weight 1 each) and its incident edge of weight $d-1$ to a clique node that is always retained. Hence, each cycle node satisfies the s -core condition in the induced subgraph consisting of the always-retained cliques together with this cycle. Therefore, the inclusion of v_j contributes an additional p nodes to the s -core.

Putting the above together, we obtain the following exact relation for every feasible A :

$$f(A) = B + (d+2)b + p \cdot \text{cov}(A).$$

Let OPT_{MC} be the optimal MC value, and let OPT_W be the optimal WASCO objective. From the relation above,

$$\text{OPT}_W = B + (d+2)b + p \cdot \text{OPT}_{\text{MC}}.$$

Assume, for contradiction, that there exists a polynomial-time $(1 - 1/e + \varepsilon)$ -approximation algorithm for WASCO. Let $\rho = 1 - 1/e + \varepsilon$, and let A be the solution returned by this algorithm. Then $B + (d+2)b + p \cdot \text{cov}(A) = f(A) \geq \rho \text{OPT}_W = \rho(B + (d+2)b + p \cdot \text{OPT}_{\text{MC}})$, which implies

$$\text{cov}(A) \geq \rho \text{OPT}_{\text{MC}} - \frac{(1-\rho)(B + (d+2)b)}{p}.$$

Since $\rho \geq 0$, we have $1 - \rho \leq 1$, so the subtracted term is at most $(B + (d+2)b)/p$. Choosing

$$p \geq \frac{2(B + (d+2)b)}{\varepsilon},$$

which is polynomially bounded in the instance size, makes this term at most $\varepsilon/2$. Hence

$$\text{cov}(A) \geq \rho \text{OPT}_{\text{MC}} - \frac{\varepsilon}{2}.$$

Since $\text{OPT}_{\text{MC}} \geq 1$ for any non-trivial MC instance, we have $\varepsilon/2 \leq (\varepsilon/2) \text{OPT}_{\text{MC}}$, and therefore

$$\text{cov}(A) \geq \left(\rho - \frac{\varepsilon}{2}\right) \text{OPT}_{\text{MC}} = \left(1 - 1/e + \frac{\varepsilon}{2}\right) \text{OPT}_{\text{MC}}.$$

This yields a polynomial-time $(1 - 1/e + \varepsilon/2)$ -approximation for MC, contradicting the inapproximability of MC unless $P = NP$ [1]. Therefore, WASCO cannot be approximated within a factor of $(1 - 1/e + \varepsilon)$ in polynomial time for any $\varepsilon > 0$, unless $P = NP$. $\square$

F Proof of Theorem 3

PROOF. Suppose that f is submodular. Then, for any two sets A and B , it must hold that $f(A) + f(B) \geq f(A \cup B) + f(A \cap B)$. Consider a graph consisting of four nodes $\{u, v, x, y\}$ and two edges (u, x) and (v, y) , each with a weight of 1. Let $s = 2$, if $A = \{(u, v), 1\}$ and $B = \{(x, y), 1\}$, $f(A) + f(B) = 0 < f(A \cup B) + f(A \cap B) = 4$. $\square$

G Proof of Property 1

Throughout, ties in follower ratio are broken by a fixed total order $\prec_{\text{tb}}$ on node pairs, e.g., the lexicographic order of vertex identifiers. This order is chosen independently of the order in which candidates are enumerated. Both GIA and GPA use $\prec_{\text{tb}}$ to resolve ties, so the greedy winner in any fixed graph state is uniquely determined.

PROOF. We prove by induction over greedy iterations that GPA and GIA select the same node pair with the same weight increment at every iteration.

Base case. Both algorithms start from the same input (G, b, s) , compute the same initial s -core $C_s(G)$, the same coreness-layer order, and the same CGI value for every candidate edge. Hence they enter the first iteration with the same state.

Inductive step. Assume that the two algorithms have selected identical edge-increment actions through the first $t-1$ iterations. Then they enter iteration t with the same current graph G_t , the same s -core $C_s(G_t)$, the same coreness-layer order, the same remaining budget, and the same CGI value for every candidate edge. Let e^* be the node

pair selected by GIA in this state under $\prec_{tb}$. We show that GPA also selects e^* with the same increment δ_{e^*} .

- (1) **Candidate reduction.** If both endpoints of e^* lie outside $C_s(G_t)$, then e^* is not affected by the reduction and is retained. Otherwise, write $e^* = (u, v)$, where $u \notin C_s(G_t)$ and $v \in C_s(G_t)$. By Theorem 5, every candidate (u, x) with $x \in C_s(G_t)$ has the same CGI value and the same follower set as (u, v) , and hence the same follower ratio. Intuitively, the only directly perturbed non-core endpoint is u ; since each $x \in C_s(G_t)$ remains stable under monotone reinforcement, the identity of the core endpoint supplying the additional support does not affect the peeling process over $V \setminus C_s(G_t)$. Thus these candidates form one tie-equivalent class. Because GPA keeps the representative of this class according to the same tie-breaking order $\prec_{tb}$, the retained representative is the same node pair that GIA would select from this class. Therefore candidate reduction does not alter the greedy choice.
- (2) **Upper-bound pruning.** Let ρ be the current best follower ratio maintained by GPA when a candidate is tested. GPA prunes a candidate only when ρ strictly exceeds a valid upper bound on that candidate's follower ratio. The candidate-skipping, partner-level termination, and global termination rules use the bounds $U(u, v)$, $U(u) + U(v)$, and $2U(u)$, respectively; by Definition 13 and the descending order of $U(\cdot)$, each bound is valid for every candidate it discards.

Suppose, for contradiction, that e^* is pruned. Let $B(e^*)$ be the bound that triggers the pruning of e^* . Then

$$FR(e^*) \leq B(e^*) < \rho.$$

Thus GPA has already evaluated a feasible candidate with follower ratio strictly larger than $FR(e^*)$, contradicting the fact that e^* is the greedy winner selected by GIA in the same graph state. Hence e^* cannot be pruned. Moreover, because the pruning condition uses a strict inequality, candidates tying the current best are retained, so the common tie-breaking order remains applicable.

- (3) **Intermediate-result reuse.** We next show that reuse preserves the true component-wise best. By Theorem 4, every follower of an anchored edge is reachable from one of its non-core endpoints by an upstairs path in G_t . Hence newly activated vertices are contained in the non-core component, or components, of $G_t[V \setminus C_s(G_t)]$ incident to the anchored edge. Consider a component K outside this affected region. If a vertex $w \in K$ were to change its support, coreness–layer pair, or upstairs reachability, then it would have to become adjacent to a newly activated s -core vertex. But such a newly activated vertex was also non-core immediately before the reinforcement; therefore, if it were adjacent to w , the two vertices would have belonged to the same component of $G_t[V \setminus C_s(G_t)]$, a contradiction. Thus the induced subgraph, boundary support from $C_s(G_t)$, coreness–layer order, and upstairs reachability relation of every unaffected component remain unchanged. GPA reuses the cached best candidate of a component only when the component is unaffected and the cached candidate remains feasible under the current remaining budget; otherwise the component is recomputed. For such an unchanged and still-feasible component, every local candidate has the same CGI value, follower set, follower ratio, and $\prec_{tb}$ -priority as when the cache was computed. Hence the cached

entry remains the true best candidate of that component. Inter-component candidates are evaluated separately under the same follower-ratio criterion and the same upper-bound pruning rules. Therefore, replacing each unchanged component's local candidate set by its cached best preserves the maximum under the lexicographic order induced by follower ratio and $\prec_{tb}$. Consequently, intermediate-result reuse cannot remove a candidate that would beat e^* , or tie with e^* and have higher tie-breaking priority.

By the three arguments above, after candidate reduction, upper-bound pruning, and safe reuse, GPA still retains the same greedy winner e^* with the same increment δ_{e^*} . Since GPA applies the same greedy criterion and the same tie-breaking order as GIA, it selects exactly the same edge-increment action in iteration t . Therefore the two algorithms enter iteration $t + 1$ with the same graph state. By induction, GPA and GIA select the same sequence of edge-increment actions and hence return the same final s -core. $\square$

H Additional Experiment

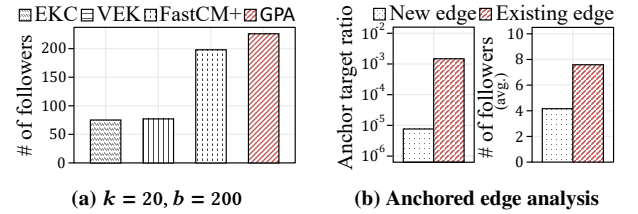


Figure 3: EQ1-4. Effectiveness on the Facebook

EQ1-4. Effectiveness (unweighted graph). We further evaluate GPA on the unweighted Facebook dataset [3] ($|V| = 4,039$, $|E| = 88,234$, $d_{avg} = 43.7$). All compared methods, including EKC [5], VEK [6], and FastCM+ [4], are originally designed for unweighted graphs. To enable a fair comparison, we treat the unweighted graph as a special case of our weighted formulation by assigning unit weight to every edge, and follow the same parameter settings ($k = 20$, $b = 200$) as in FastCM+. As shown in Figure 3(a), GPA can achieve the largest number of followers among all compared methods under the same budget. This result indicates that, even when restricted to the unweighted setting, GPA is capable of operating effectively in practice. Figure 3(b) further analyses the anchoring behaviour of GPA. After normalising by the size of the corresponding feasible action spaces, edge reinforcement is selected more frequently than new edge insertion. Moreover, reinforcement actions tend to yield a higher average number of followers per anchored edge. These observations suggest that strengthening existing connections can be a more effective anchoring strategy than edge insertion, even when all edges have uniform weight.

References

- [1] Uriel Feige. 1998. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM (JACM)* 45, 4 (1998), 634–652.
- [2] Samir Khuller, Anna Moss, and Joseph Seffi Naor. 1999. The budgeted maximum coverage problem. *Information processing letters* 70, 1 (1999), 39–45.
- [3] Jure Leskovec and Julian McAuley. 2012. Learning to discover social circles in ego networks. *Advances in neural information processing systems* 25 (2012).

- [4] Xin Sun, Xin Huang, and Di Jin. 2022. Fast algorithms for core maximization on large graphs. *Proceedings of the Very Large Data Bases (VLDB) Endowment* 15, 7 (2022), 1350–1362.
- [5] Zhongxin Zhou, Fan Zhang, Xuemin Lin, Wenjie Zhang, and Chen Chen. 2019. k -Core Maximization: An Edge Addition Approach.. In *International Joint Conference on Artificial Intelligence (IJCAI)*. 4867–4873.
- [6] Zhongxin Zhou, Wenchao Zhang, Fan Zhang, Deming Chu, and Binghao Li. 2022. VEK: a vertex-oriented approach for edge k -core problem. *World Wide Web* 25, 2 (2022), 723–740.